

PROJECT REPORT ON

“EduQuery AI”



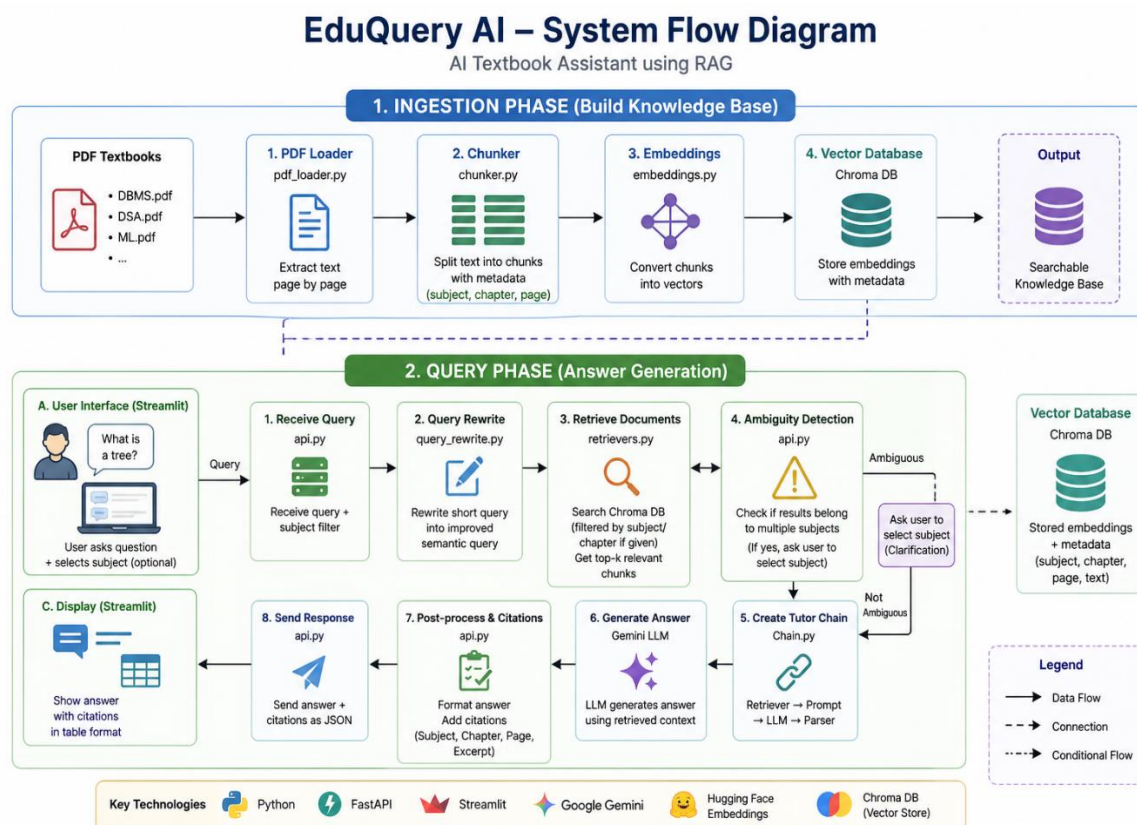
Project Overview

EduQuery AI is an intelligent textbook question-answering system designed to help students retrieve accurate information from multiple academic subjects such as DBMS, Data Structures, and Machine Learning.

The system is built using a Retrieval-Augmented Generation (RAG) architecture, which combines semantic search with a Large Language Model (LLM) to generate context-aware answers based strictly on textbook content.

Project Goals

The goal of EduQuery AI is to build an AI-powered textbook assistant that provides accurate, context-based answers using a Retrieval-Augmented Generation (RAG) approach. It aims to enable semantic search across multiple subjects, improve query understanding, handle ambiguous queries, and deliver transparent responses with citations through a user-friendly chat interface.



Requirements

◆ Project Configuration

PROJECT_NAME: str = "EduQuery AI"

VERSION: str = "1.0.0"

requires-python = ">=3.9"

MODEL_NAME: str = "gemini-3-flash-preview"

EMBEDDING_MODEL: str = "sentence-transformers/all-MiniLM-L6-v2"

◆ Project Dependencies

```
dependencies = [  
    "fastapi>=0.100.0",  
    "uvicorn>=0.22.0",  
    "streamlit>=1.25.0",  
    "requests>=2.31.0",  
    "pandas>=2.0.0",  
    "python-dotenv>=1.0.0",  
    "pymupdf>=1.24.0",  
  
    "langchain>=0.1.0",  
    "langchain-core>=0.1.0",  
    "langchain-text-splitters>=0.1.0",  
    "langchain-google-genai>=1.0.0",  
    "langchain-huggingface>=0.0.3",  
    "langchain-chroma>=0.1.0",  
  
    "chromadb>=0.5.0",  
    "sentence-transformers>=2.7.0"  
]
```

EduQuery AI – Project Structure

Project Structure	Description / Purpose
- EduQuery_AI/ - chroma_db/ - data/ - pdf/ *.pdf - src/ - pdf_loader.py - chunker.py - embeddings.py - retrievers.py - query_rewrite.py - Chain.py - api.py - model.py - streamlit_app.py - ingest_all.py - requirements.txt - README.md	Stores vector embeddings (acts as system memory / knowledge base). Contains all project data. Stores all textbook PDF files. Textbook PDF files (DBMS, DSA, ML, etc.). Contains all core source code. Extracts text from PDFs page-by-page using PyMuPDF. Splits extracted text into chunks and adds metadata (subject, chapter, page). Creates embeddings for text chunks and manages ChromaDB operations. Performs semantic search on ChromaDB with optional filters (subject, chapter). Rewrites short or unclear user queries to improve retrieval accuracy. Builds the RAG pipeline (retriever + prompt + Gemini model) and generates answers. FastAPI backend. Handles upload, query processing, ambiguity detection and returns responses with citations. Initializes and provides access to Gemini LLM and embedding model. Frontend application using Streamlit. Provides chat interface and displays answers with citations. Runs the complete ingestion pipeline. Processes all PDFs and builds the vector database. Lists all Python dependencies required to run the project. Project overview, setup instructions, usage guide and other details.

Project Files Explanation

◆ chroma_db/

This folder stores vector database files.

- Contains embeddings + metadata
- Persistent storage

👉 Purpose: System memory

◆ data/pdf/

This folder contains input textbooks.

👉 Purpose: Raw data source

◆ pdf_loader.py

This file is responsible for reading textbook PDFs and extracting text.

- Uses PyMuPDF (fitz) to open PDF files
- Reads each page one by one
- Extracts text and removes empty content
- Returns structured data:
 - page text
 - page number

👉 Purpose: Convert raw PDF → usable text

◆ chunker.py

This file splits large text into smaller pieces (chunks).

- Uses RecursiveCharacterTextSplitter
- Detects chapters using regex
- Splits text into chunks (size: 500, overlap: 100)
- Adds metadata:
 - subject
 - chapter
 - page

👉 Purpose: Make text searchable and meaningful for AI

◆ embeddings.py

This file handles vector database operations.

- Converts text chunks into embeddings (vectors)

- Uses HuggingFace model
- Stores embeddings in ChromaDB
- Loads existing database if available

👉 Purpose: Enable semantic (meaning-based) search

◆ **ingest_all.py**

This file runs the ingestion process.

- Loads all textbooks
- Converts them into chunks
- Stores them in vector database

👉 Purpose: Build the knowledge base

◆ **retrievers.py**

This file is responsible for fetching relevant data.

- Connects to vector database
- Retrieves top-k similar chunks
- Supports filtering:
 - subject
 - chapter

👉 Purpose: Find the most relevant content for a query

◆ **query_rewrite.py**

This file improves user queries.

- Uses Gemini model

- Converts short/vague queries into detailed ones
- Keeps original meaning

👉 Example:

- "tree?" → "Explain tree data structure"

👉 Purpose: Improve search accuracy

◆ Chain.py

This file builds the AI pipeline (RAG system).

- Combines:
 - Retriever
 - Prompt template
 - Gemini model
- Formats context + question
- Generates structured output (FOUND / NOT_FOUND)

👉 Purpose: Generate final answer using context

◆ api.py

This is the main backend (FastAPI server).

Handles:

- /upload → upload and process PDFs
- /ask → answer user queries

Key features:

- Query rewriting

- Ambiguity detection
- Retrieval + answer generation
- Citation creation

👉 Purpose: Acts as the brain of the system

◆ **model.py**

This file loads AI models.

- Loads Gemini LLM
- Loads embedding model

👉 Purpose: Central place for model configuration

◆ **streamlit_app.py**

This file creates the frontend UI.

- Chat interface for user interaction
- Sends query to FastAPI
- Displays:
 - answers
 - citations
- Maintains chat history

👉 Purpose: User interface of the system

◆ **requirements.txt**

Contains all required Python libraries.

👉 Purpose: Setup environment easily

◆ README.md

Project documentation.

👉 Purpose: Explain project, setup, usage

Output:

The screenshot displays the user interface of the 'EduQuery AI: Student Textbook Assistant'. On the left, a sidebar contains a 'Filter Subject' dropdown menu set to 'All' and a 'Clear History' button. The main content area features the title 'EduQuery AI: Student Textbook Assistant' with a graduation cap icon. Below the title, there are two question-and-answer pairs. The first question is 'What is ML', followed by an answer explaining that Machine Learning (ML) is a subset of Artificial Intelligence that enables systems to learn from data and improve their performance over time without being explicitly programmed. The second question is 'What is tree', followed by an answer explaining that in the context of supervised learning, a Decision Tree is an algorithm that recursively splits data based on the feature that provides the maximum information gain or minimum Gini impurity. At the bottom of the main area is an input field labeled 'Ask Questions...' with an upward arrow button. In the top right corner of the interface, there is a 'Deploy' button and a vertical ellipsis menu icon.

Filter Subject

DBMS

Clear History

Deploy



EduQuery AI: Student Textbook Assistant



Anamolies



In the context of unnormalized tables, anomalies are issues that occur during data manipulation. There are three specific types identified:

- **Insertion Anomaly:** This occurs when data cannot be inserted without providing unrelated data. For example, you cannot add a new course to the database unless a student enrolls in it.
- **Deletion Anomaly:** This occurs when deleting one fact causes the loss of another unrelated fact. For example, deleting the last student enrolled in a course results in the removal of the course information itself.
- **Update Anomaly:** This occurs when changing one fact requires multiple row updates. For example, updating a professor's department would require many changes across different rows to maintain consistency.



Source Citations

Ask Questions...

